Getting Started in C

CSC 230 : C and Software Tools

NC State Department of Computer Science

Topics for Today

- C Overview
- Software Tools
- The common platform
- Building a program
- The C you already know

You **Want** to Learn C

- It's a lower-level language than Java
  - Can offer much better performance (often not that important)
  - Exposes more of what's going on underneath
  - This can make us more effective developers (even when we're working in a higher-level language)
- It will help us:
  - Prepare for CSC 246 (Required, Operating Systems)
  - Prepare for CSC 236 (Elective, Assembly Language)
  - Learn other programming languages
  - Develop for other environments

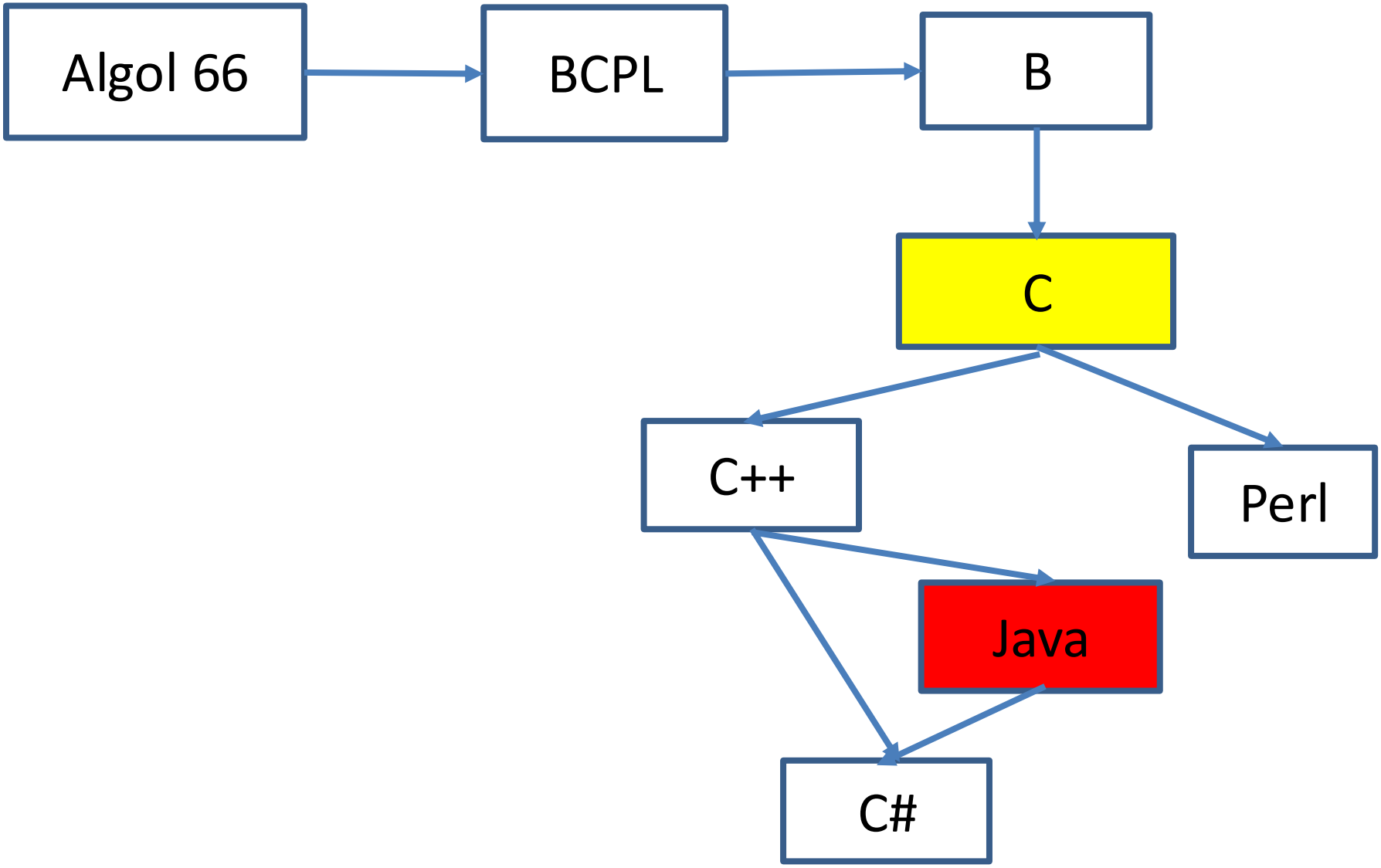
You Want to Learn C

- Someone has to be able to program in C
  - That operating system you like to use, what do you think it's written in?
    - Linux : Assembly and C
    - MS Windows : Assembly, C and C++
  - How about that JVM that lets you run your fancy, high-level programs?
    - Assembly, C and C++
  - Lots of other examples
    - Embedded systems (cars, calculators, appliances, etc.)
    - High-performance applications (science/engineering)

Thinking in C

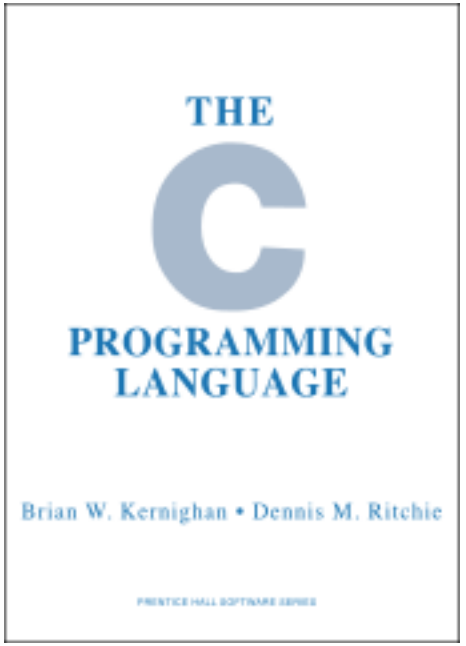
- C : Programming in a different Type of language
  - Like Java, it's an *imperative language*, focused on **how** a computation should be performed
  - C is *procedural*
    - A program is a collection of procedures (functions)
    - Focus on **actions** performed by these procedures
  - Instead of *object-oriented*
    - Where program is a collection of objects, each with state and operations
    - Focus on the **state** of these objects
- Of course, there are other ways we could go …
  - *Declarative Languages* : focus on ***what*** the program should compute rather than ***how*** it should compute it
    - *Functional Languages*: Lisp, Scheme, Haskell
    - *Dataflow Languages*: VHDL, Linda, TensorFlow
    - *Logic* or *Constraint-Based*: Prolog
    - *Markup Languages*: HTML, Markdown

How We Got Here



How We Got Here

- Developed along with the Unix operating system
  - An alternative to developing the OS in assembly
    - More portable
    - More readable/maintainable
- What's C
  - Informal standard for 10 years
  - C89 standard in 1989/1990
  - C99 standard in 1999
  - C11 completed in … 2011
  - C17 (2017), C23 (2023)
  - Work on C2Y is ongoing



C Strengths and Weaknesses

- Think of C as a thin veneer over the underlying assembly language
  - Lets us do some things we couldn't do in a higher-level language ☺
  - The standard leaves some details implementation-defined, to better exploit the hardware. For example:
    - C has an int type, but its range may depend on compiler / hardware.
    - C doesn't generally guarantee what uninitialized variables will contain.
- C programs and the compiler can be tiny and fast ☺
  - Example trivial C and Java programs
  - Example sorting
  - Example matrix multiplication

C Performance vs Java

- C is faster at nothing:

```
$ time java Nothing

real    0m0.138s
```

```
$ time ./nothing

real    0m0.002s
```

About 70 times faster.

- Also, faster at sorting 60,000 words

```
$ time java SortWords words.txt

real    0m1.529s
```

```
$ time ./sortWords words.txt

real    0m0.063s
```

About 24 times faster.

C Performance vs Java

- Or, multiplying 400 X 400 matrices.

```
$ time java Matrix ab.txt

real    0m3.458s
```

```
$ time ./matrix

real    0m0.601s
```

About 6 times faster.

- Why so much faster?
  - Well, C is compiled down to the hardware's native machine language.
  - And, there are a lot of things Java does that C doesn't

C Strengths and Weaknesses

- C offers very little in *protection* and *security* ☹
- C lacks some constructs for managing very large projects ☹

It’s Not Just About C

- Software tools
  - To help write, build, analyze and maintain software
  - Coordinate contributions from a team
- Examples:
  - Editors, pretty printers
  - Compilers, linkers
  - Debuggers
  - Code generators
  - Performance analyzers
- Often, these are integrated into the IDE
  - But, there’s some value in being able to run them directly

The Common Platform

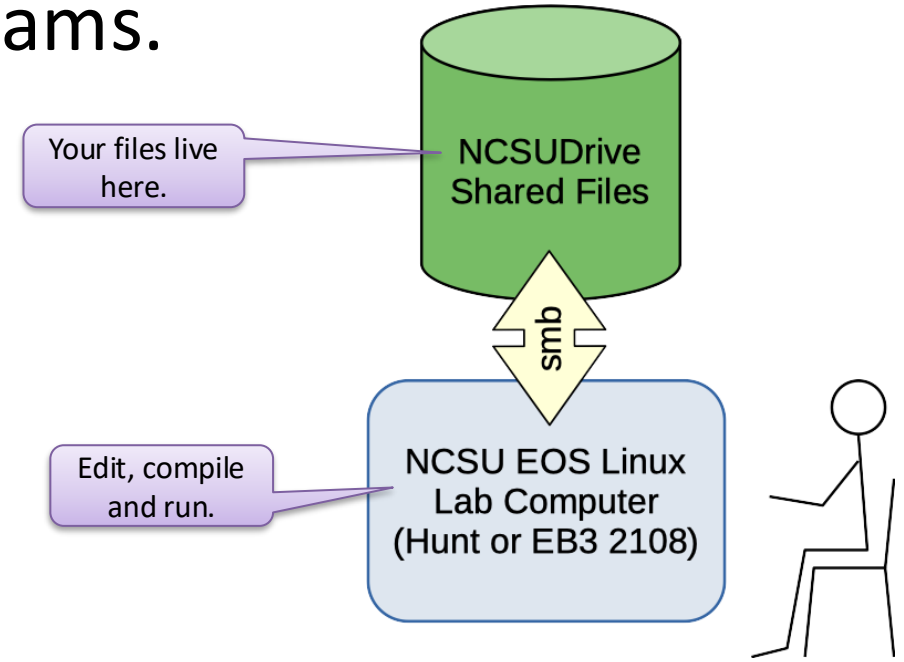
- Different systems have different processors, compilers, language versions, line termination conventions, etc.
- We need to agree on what system to target
- We call this is our *common platform*
  - 64-bit Intel PC
  - University Linux OS
  - gcc compiler suite
- Readily available on campus and from home

Working in C

- You have some choices in how you work
- EOS Linux machines (Common Platform)
  - These are the kinds of systems we'll test your work with
- Your own system with a C Compiler (Not Common Platform)
  - These can let you get work done … but
  - you may not have access to all the tools you need.
  - you should **test on a real EOS Linux system** before submitting your work.

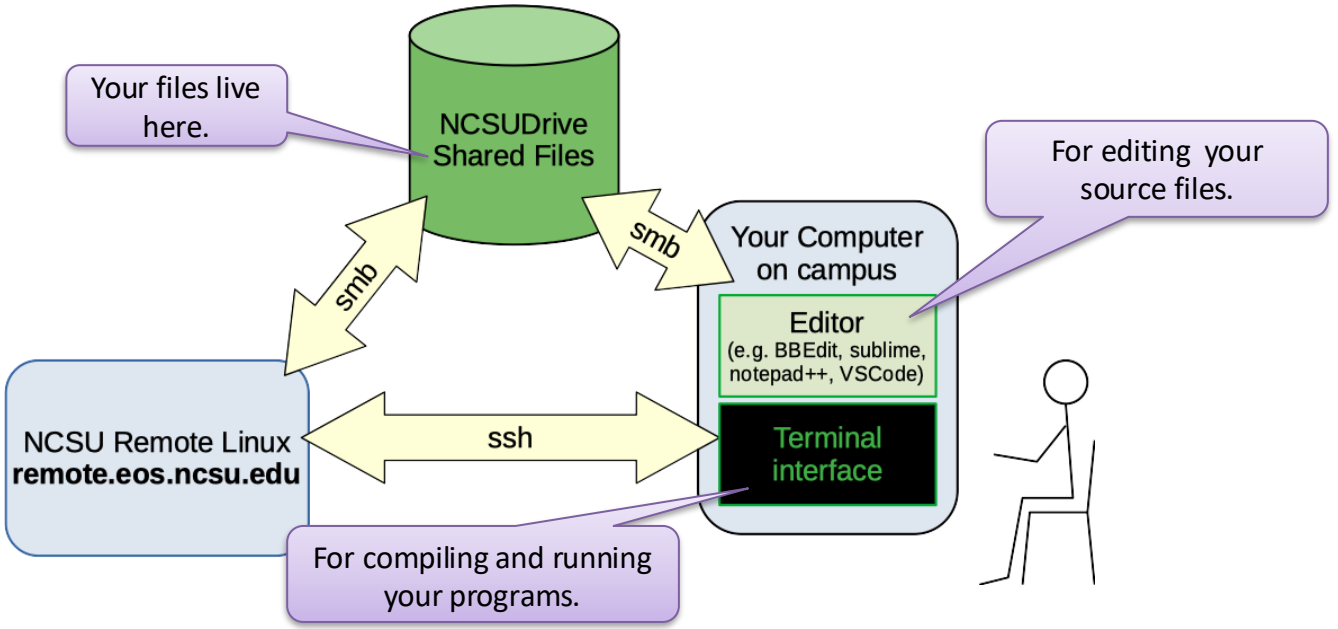
EOS Lab Systems

- Available in Hunt Library, EB3, etc.
- Access to files in NCSU Drive
- Linux Desktop for Terminal and Graphical Programs.



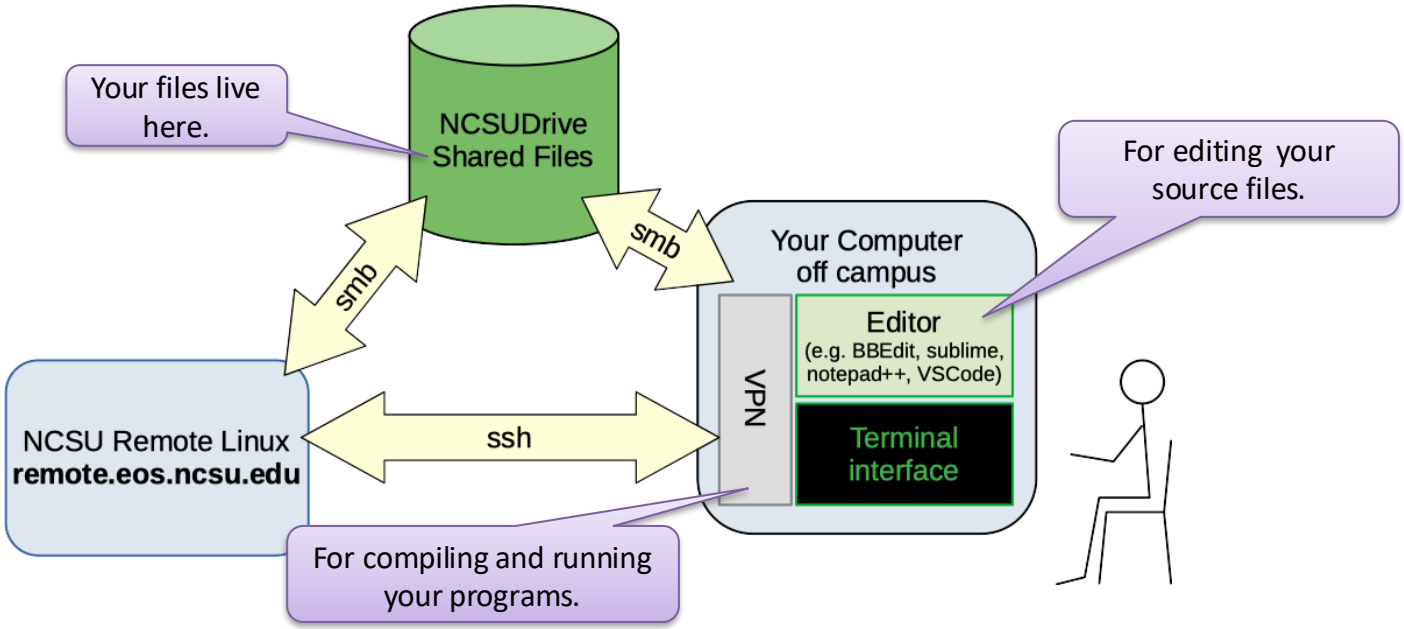
On-Campus Access to NCSU Drive and Remote Linux Systems

- Available from anywhere on campus
  - Access to files in NCSU Drive
  - Terminal interface for compile / execute
  - Text editor on your own laptop



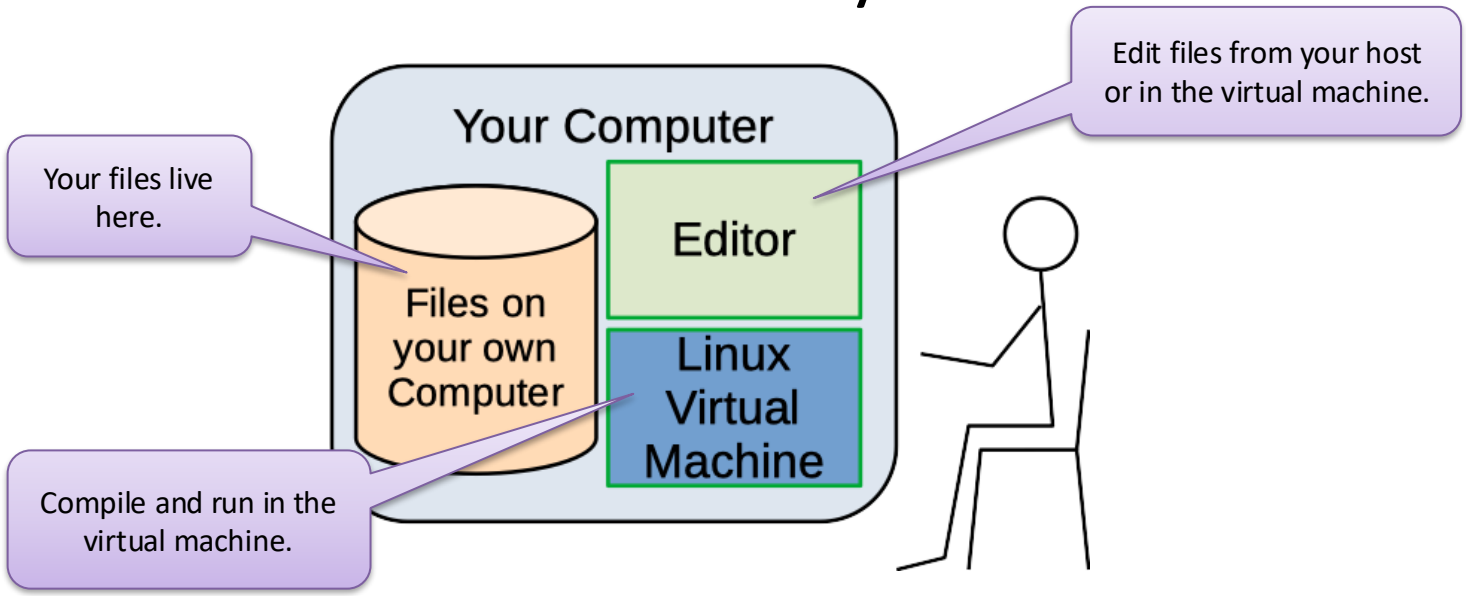
Off-Campus Access with VPN

- Virtual Private Networking (VPN)
- Lets you access NCSU Drive from off-campus



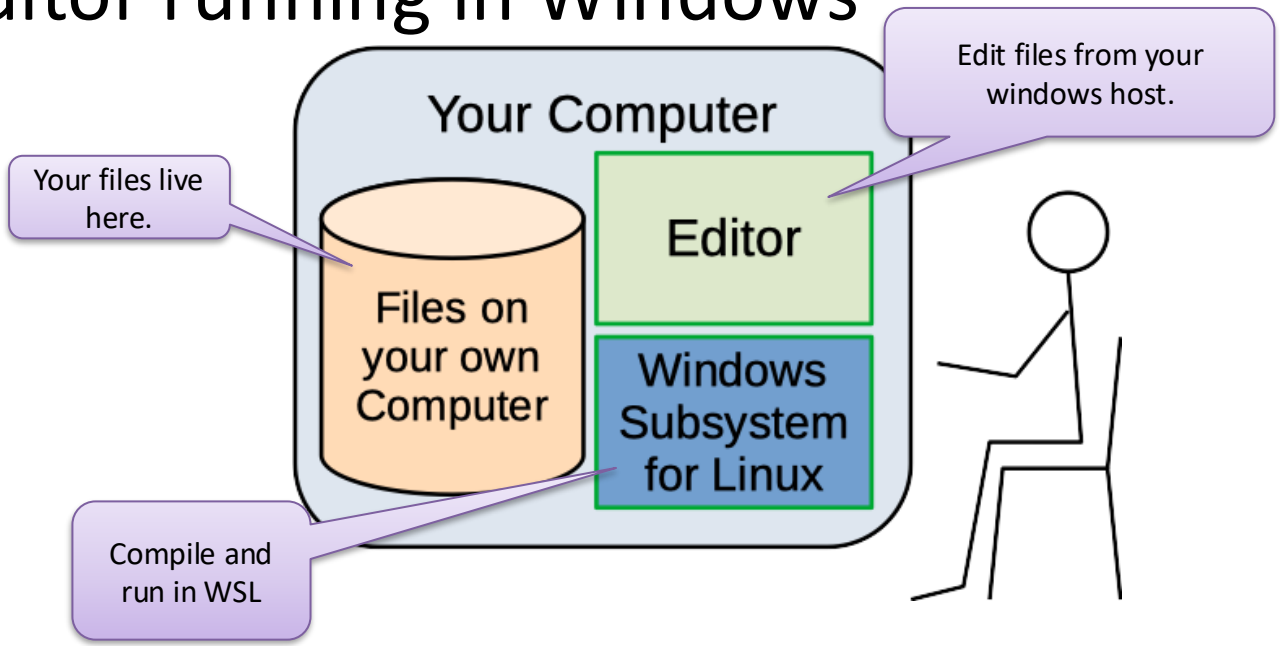
Linux in a Virtual Machine

- Everything on your laptop / desktop
- Environment similar to EOS Linux
  - We have a pre-made VM Image if you want to use it.
  - But not a perfect Common Platform Installation
- Can install other software you need.



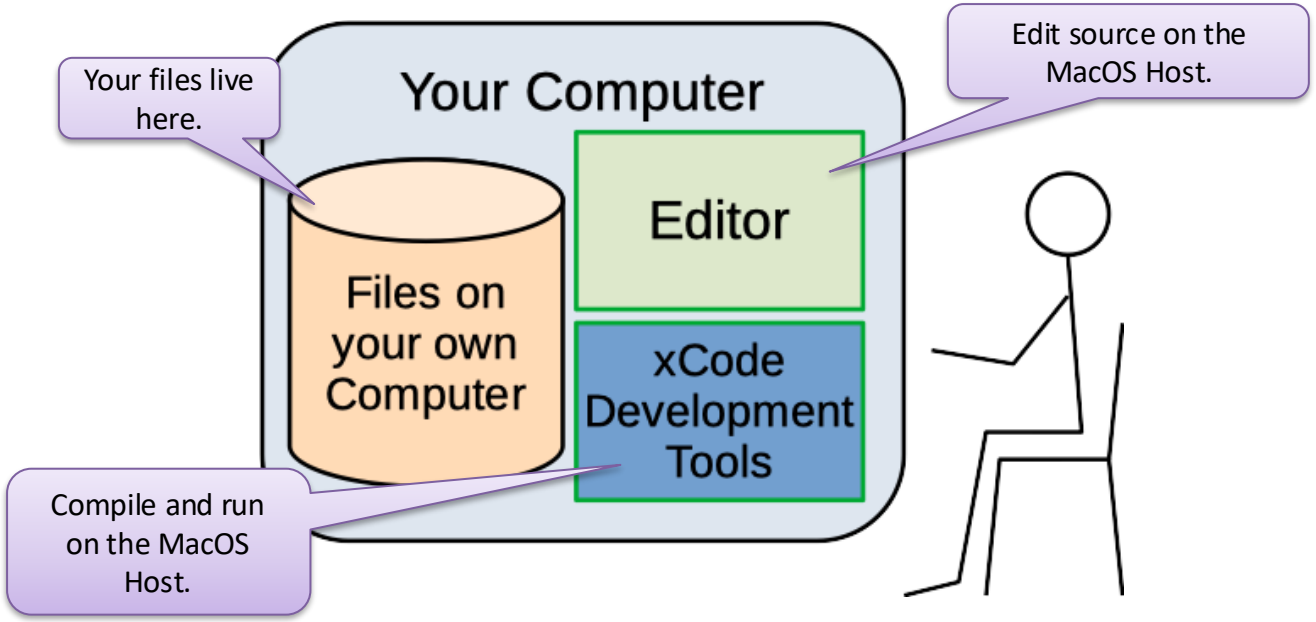
Windows Subsystem for Linux

- Everything on your desktop
- Linux installation for command line
  - But not a perfect Common Platform Installation
- Editor running in Windows



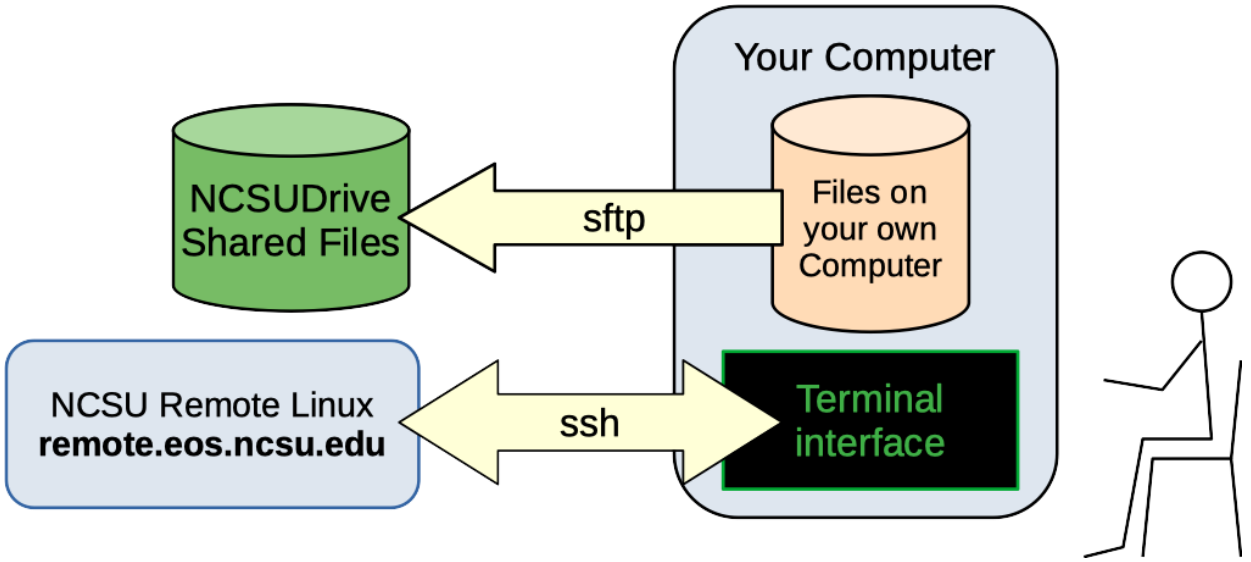
xcode on MacOS

- Everything on your desktop
- Compiler supports gcc options
  - But not a real common platform system
  - Missing some development tools
- Editor and other tools running in MacOS



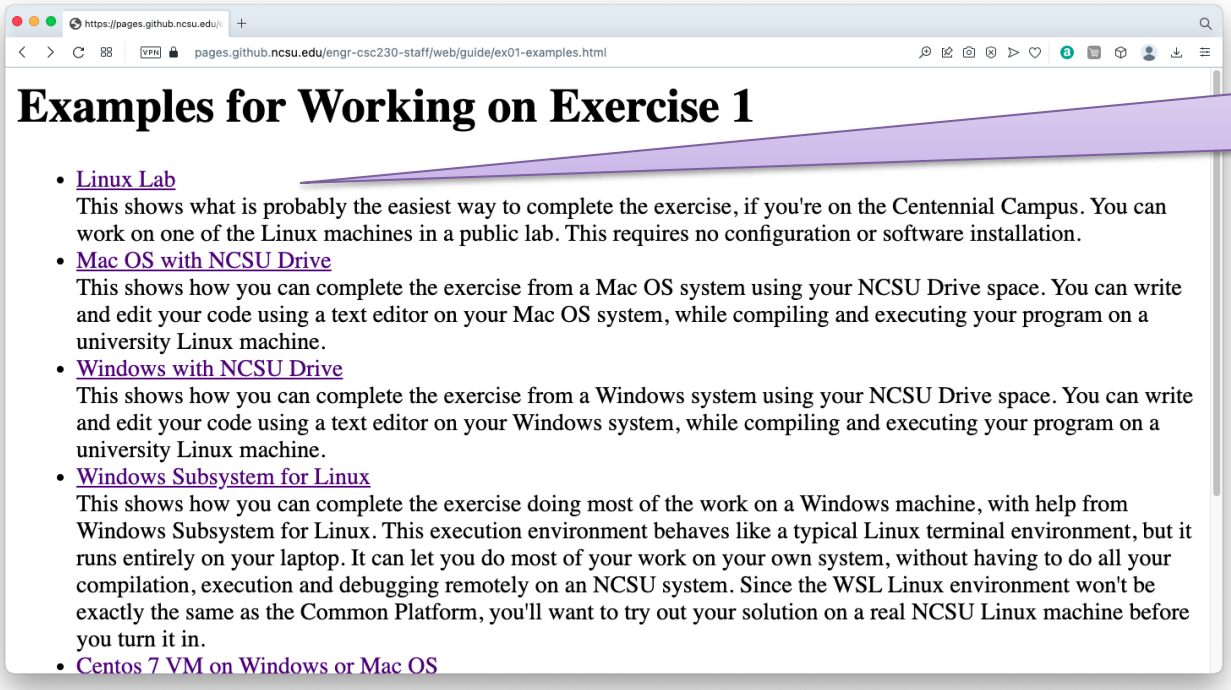
Checking your Solution

- If you develop on your own system …
  - It's a good idea to check your work on a real Common Platform system before you submit
  - There can be some differences from one system to another
  - … even Linux systems
  - ... especially if you have some bugs.
- Copy your solution to an EOS Linux System (e.g., remote.eos.ncsu.edu)
- Compile and test it there.



Course Support

- Our Moodle site describes many of these options for working on CSC 230 Assignments.



Meet C

```
/**
  @file hello.c
  @author David Sturgill (dbsturgi)
  A program that prints: Hello World
 */
#include <stdio.h>

/**
  Starting point for the program.
  @return exit status
 */
int main()
{
  printf( "Hello World\n" );
  return 0;
}
```

Meet C

```
/**
  @file hello.c
  @author David Sturgill (dbsturgi)
  A program that prints: Hello World
 */
#include <stdio.h>

/**
  Starting point for the program.
  @return exit status
 */
int main()
{
  printf( "Hello World\n" );
  return 0;
}
```

Compile like this

```
$ gcc -Wall -std=c99 hello.c -o hello
$ ./hello
```

Execute like this

What are You Looking At?

```
/**
  @file hello.c
  @author David Sturgill (dbsturgi)
  A program that prints: Hello World
 */
#include <stdio.h>

/**
  Starting point for the program.
  @return exit status
 */
int main()
{
  printf( "Hello World\n" );
  return 0;
}
```

A Comment, part of our style requirements

Telling the compiler about library components we use below (just printf).

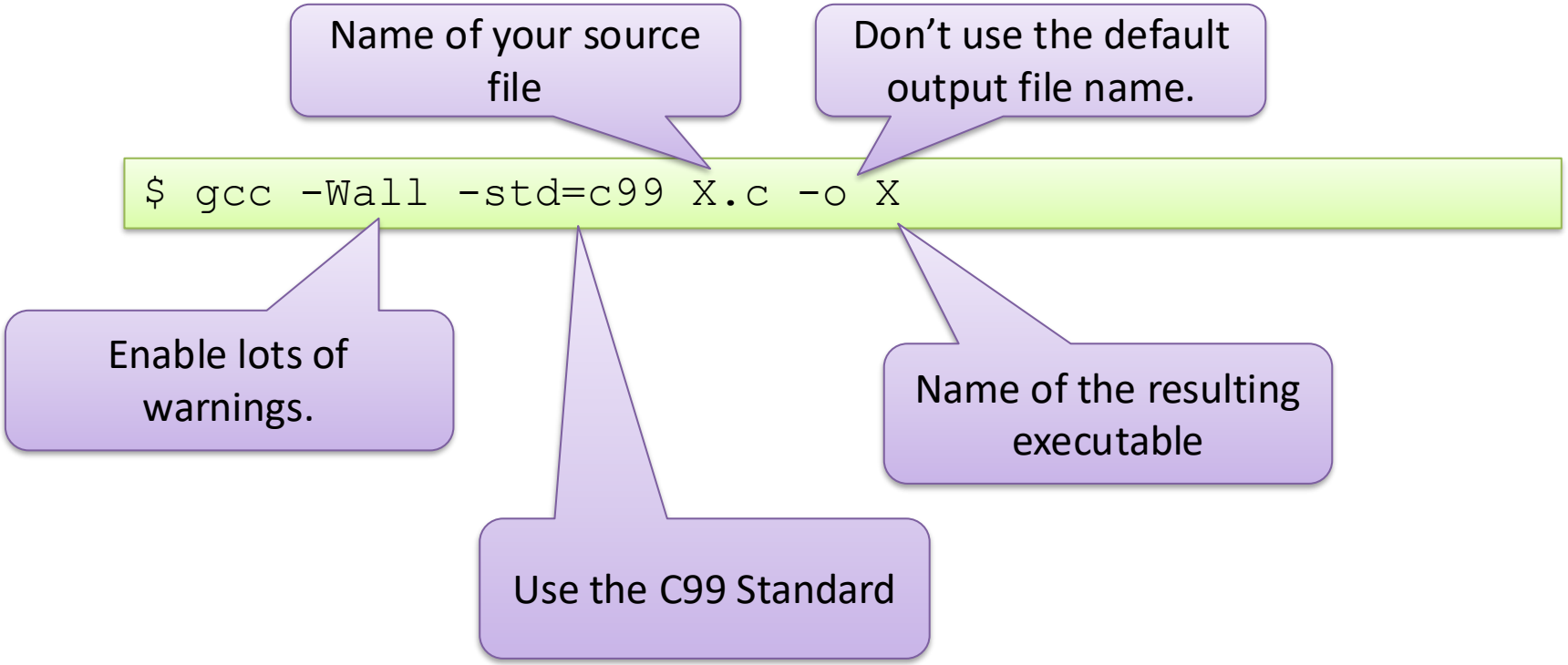
A main function, where your program starts (see, it's not inside a class).

A call to the printf function to, well, print something out.

We're all done. Exit with success.

Building C Programs

- Here's a recipe for building any simple C program:



The C You Already Know

- Some parts of the C language look a lot like Java
  - Variable declarations look the same

```
int main()
{
    int a = 25;
    double x = 3.14;
    char c = '*';
    .
    .
}
```

Before the C99 standard, local variables had to be declared at the top of a function or block.

Some people still code that way.

  - We have most of the same built-in types, including **char**, **short**, **int**, **long**, **float** and **double**
  - No **byte** and **boolean** types, though.

We use char instead.

C99 Gives us a bool type.

The C You Already Know

- C has many of the operators you remember from Java … and they mostly work the same.

Arithmetic	+, -, *, /, %
Logical	!, &&, ||
Pre-increment, etc.	++, --
Relational	==, !=, <, >, <=, >=
Assignment	=, +=, *=, etc.

Looks just like good old Java.

But, C has some operators Java doesn't have.

```
int a = 25;
double x = a * 1.5;

a++;
x = x / 2;
x += a % 3;

int *p = &a;
```

The C You Already Know

- Flow of control
  - C has an if statement that looks a lot like Java:

```
if ( a > 25 ) {
    a /= 2;
}
```

  - … and a for loop (that looks a lot like Java):

```
for ( int i = 0; i < 25; i++ ) {
    x += i;
}
```

The C You Already Know

- … and a while statement (that looks a lot like Java):

```
while ( x < 100.0 ) {
    x *= 1.05;
}
```

- … and a do/while:
  (that's basically the same as Java)

```
do {
    x = getchar();
} while ( x != '\n' );
```

The C You Already Know

– … and a switch statement:
(that's more restrictive than Java's switch)

This has to be an integer.

```
switch ( i ) {
    case 0:
        print( "it's zero!\n" );
        break;
    case 1:
        …;
    default:
        …;
}
```

The C You Already Know

- Flow of control
  - And C has break and continue statements

.. that you should avoid using when you can.

  - … and one other thing Java doesn't have.

.. that will help you appreciate why you should avoid break & continue.

The C You Already Know

- Functions
  - C has functions, with parameters and return types

```
double power( double x, int p )
{
    double result = 1;
    for ( int i = 0; i < p; i++ )
        result *= x;
    return result;
}
```

Here's a function definition.

```
y = power( 3.25, j + 1 );
```

Here's a function call.

  - Like static methods in Java

The C You Already Know

- Some things are different.
  - C functions aren't part of a class, they are defined at *file scope*

```
double power( double x, int p )
{
    double result = 1;
    for ( int i = 0; i < p; i++ )
        result *= x;
    return result;
}
```

```
    y = power( 3.25, j + 1 );
```

And, C99 wants to see the function definition (or declaration) **before** you try to call it.

Not Quite Java

- **C doesn't force you to initialize your local variables.**
- **… and, it doesn't initialize them for you.**

```
double power( double x, int p )
{
    double result;
    for ( int i = 0; i < p; i++ )
        result *= x;
    return result;
}
```

```
y = power( 3.25, j + 1 );
```

Oops.
What value will this variable have?
Whatever was in that region of memory.

What will this return?
Probably garbage.